This example demonstrates another feature of RSpec's dynamic predicate matchers: they support arguments and a block. If you pass arguments (or a block) to a dynamic predicate matcher, RSpec will forward them on to the predicate method when it calls it on the expectation subject.

Trade-offs

As readable and useful as dynamic predicate matchers can be, they do have some trade-offs. Like the truthiness matchers, the dynamic predicate matchers use Ruby's loose Boolean conditional semantics. Most of the time, this is what you want, but you'll need to use a different technique if you want to test for exact `true` or `false` results.

A bigger problem is documentation. Because dynamic matchers are generated on the fly, they have no documentation. Your teammates may see `be_an_admin` in your tests and go looking for it in vain in the RSpec docs. A simple equality matcher wouldn't have this problem:

11-matchers-included-in-rspec-expectations/01/primitive_matchers.rb

```
expect(user.admin?).to eq(true)
```

However, the output from the simple matcher isn't nearly as helpful:

```
expected: true
     got: false

(compared using ==)
```

The failure output from using `be_an_admin` is far better:

```
expected `#<User name="Daphne">.admin?` to return true, got false
```

Dynamic predicate matchers are useful in composed matcher expressions. For instance, you can customize how a collection matches based on a predicate:

11-matchers-included-in-rspec-expectations/01/primitive_matchers.rb

```
expect(array_of_hashes).to include(have_key(:lol))
```

The language is a little stilted here, since dynamic matchers have no built-in aliases. If you want a nice, readable noun-phrase version of a predicate matcher, you'll have to alias it yourself. Luckily, it's trivial to do so, as we'll show in *Defining Matcher Aliases*, on page 218.

Given these trade-offs, there's no slam-dunk case for or against them. Personally, we like them, but talk it over with your team before you strew these all over your specs.